ONEDESK

FINAL MASTER DEVELOPMENT BLUEPRINT

A project-centric, role-based work management and team collaboration platform.

ONE WORKSPACE. EVERY PROJECT. ONE CLEAR VIEW.

DOCUMENT	EDITION	TARGET
Product Requirements + Engineering Specification	v2.0 | October 2026	Google Antigravity / Engineering Team

Covers product scope, complete user journeys, roles and permissions, project isolation, 10 functional systems, 45-screen inventory, data model, UI design, implementation phases, quality gates, and copy-ready build instructions.

Design direction: Alpine Sage + Obsidian Forest / Minimal / Responsive

01 Executive Product Brief

OneDesk is a multi-tenant SaaS workspace where individuals and teams organize projects, assign work, collaborate in real time, and track progress. Its defining experience is project-level independence combined with permission-aware centralized visibility.

> **Core promise: A workspace owner can create multiple projects, assign a different team and role to each project, communicate separately with each team, and see every authorized project on one dashboard.**

Primary audiences

- Agency owners managing parallel delivery projects.
- Startup founders coordinating design, development, operations, and launch work.
- Project managers overseeing deadlines, reviews, blockers, and workload.
- Contributors completing assigned tasks across one or more projects.
- Individuals planning personal tasks, goals, and documents.

Explicit scope exclusions

Do NOT build CRM, client portals, lead management, sales pipelines, invoicing, accounting, payments, subscriptions, billing, revenue analytics, or financial tracking. Do not introduce these concepts as placeholders or navigation items.

Non-negotiable product principles

- One user may belong to multiple workspaces, with different workspace roles.
- One user may belong to multiple projects, with different project roles.
- Private project tasks, files, and channels must not leak across projects.
- Every data view, API, search result, notification, and AI context is permission-aware.
- All major actions persist in the database and synchronize across relevant views.
- No fake metrics, decorative-only buttons, or mock functionality counted as done.

02 User Personas and End-to-End Journeys

Journey A: Agency owner

- Sign up, verify email, create a workspace named Factor Labs.
- Create Project A (Real Estate Website), Project B (Mobile App), Project C (Brand Identity).
- Invite 8 teammates to the workspace, then assign project-specific memberships and roles.
- Create project tasks, dependencies, milestones, due dates, and default channels.
- Assign tasks to contributors; each person sees work from authorized projects.
- Review submissions, approve or request rework, and inspect project activity.
- Read project-specific chat and receive unified notifications across projects.
- Open the Owner Command Center to monitor project health, overdue work, workload, approvals, and milestones.

Journey B: Project manager

- Open managed projects, create task breakdown, assign owners, and set deadlines.
- Track status, blockers, dependencies, and sprint commitments.
- Review submitted work and request changes or approve according to configured policy.
- Communicate through project channels and task comments.

Journey C: Contributor

- Log in to My Work: Today, Upcoming, Assigned, In Review, and Mentions.
- Work on tasks in authorized projects, attach files, update status, log time.
- Submit for review, receive feedback, and resolve requested changes.
- Never see unauthorized private project content.

Journey D: Personal user

Create a personal workspace, projects, tasks, calendar entries, goals, notes, and time entries without inviting a team. Team features remain available when needed.

03 Information Architecture and Data Boundaries

> **User → Workspace Membership → Workspace → Space → Optional Folder → Project → Task → Subtask / Checklist**

Projects can live directly under a Space or inside a Folder. A project is the central collaboration unit. A workspace can also host shared channels, documents, goals, and dashboards that are not attached to a single project.

Project boundary rules

- Each project has its own members, roles, task board, chat channels, files, docs, milestones, settings, and activity.
- A user can be a Project Manager in Project A and Contributor in Project B.
- Workspace membership is required for project access unless a specifically designed, secure guest-sharing model permits access.
- A private project requires explicit project membership or narrowly scoped authorized access.
- Project content is never exposed simply because a user is a platform admin or a workspace admin.
- Project deletion/archival preserves auditability and applies defined retention and restore rules.

04 Roles, Permissions, and Governance

Administration levels

Scope	Role	Allowed scope
Platform	Super Admin / Operator	Platform health, user safety, feature flags, operational logs; no blanket private-content access.
Workspace	Owner	Workspace settings, membership, ownership, and authorized resources.
Workspace	Admin	Delegated workspace administration; not ownership transfer unless explicitly granted.
Workspace	Member / Guest	Access based on workspace membership and resource permissions.
Project	Owner	Project lifecycle, members, settings, and permissions.
Project	Manager	Planning, assignments, reviews, progress, and authorized moderation.
Project	Contributor	Edit assigned or permitted work, comment, submit, collaborate.
Project	Viewer	Read only authorized project data.

Important distinction

Job titles (Frontend Developer, Designer, QA, Writer) describe responsibilities. They are NOT authorization roles. Keep project_role and job_title separate.

Permission matrix - minimum requirements

Action	Owner	Manager	Contributor	Viewer
Manage project settings	Yes	Delegated	No	No
Invite / remove project members	Yes	If allowed	No	No
Create / assign tasks	Yes	Yes	Configured	No
Update assigned task	Yes	Yes	Yes	No
Submit work for review	Yes	Yes	Yes	No
Approve required review	If reviewer	If reviewer	Only if explicitly allowed	No
View project channels	By channel ACL	By channel ACL	By channel ACL	By channel ACL
Read private unrelated project	No by default	No	No	No

Security invariant

Never rely on hidden buttons or route guards alone. Authorization must be enforced at the database/API boundary and rechecked for every mutation, file download, search result, realtime subscription, and AI retrieval.

05 Ten Functional Systems

01. Identity & Workspaces

- Email/password and OAuth authentication; verification, reset, secure sessions.
- Workspace creation, switching, settings, ownership transfer.
- Invitations, membership status, workspace roles, guest restrictions.
- Account profile, preferences, notification settings, account deletion.

Acceptance: A user can join two workspaces without cross-tenant data exposure.

02. Projects & Task Engine

- Spaces, folders, projects, templates, project members and roles.
- Task CRUD, subtasks, checklists, tags, custom fields, statuses, priorities.
- Assignees, watchers, due dates, recurring tasks, dependencies, blockers.
- Task comments, attachments, history, archive, trash, restore.
- Configurable approval workflow and reviewer assignments.

Acceptance: A task update persists, is authorized, and is visible consistently in all project views.

03. Views & Analytics

- List, Kanban, Table, Calendar, Gantt, Timeline, Workload.
- Saved views, filters, sorting, grouping, bulk operations.
- Project dashboards and owner cross-project command center.
- Progress, overdue work, capacity, activity, milestone and sprint metrics.

Acceptance: All charts reflect authorized persisted data, never hardcoded demo numbers.

04. Team Chat & Collaboration

- Project channels, workspace channels, group and direct messages.
- Threads, reactions, mentions, pins, search, attachments.
- Read state, typing indicators, online presence where appropriate.
- Task-linked messages and per-channel membership rules.

Acceptance: A private project channel cannot be queried by a nonmember.

05. Documents & Files

- Rich-text documents, wiki, nested pages, templates.
- Autosave, version history, comments, mentions, collaboration.
- Project docs, task links, file uploads, previews, access control.

Acceptance: Concurrent document edits do not silently overwrite each other.

06. Goals, Sprint, Calendar & Time

- Goals, OKRs, milestones, backlog, sprints, story points.

- Calendar, reminders, recurring events, planning and roadmaps.
- Time estimates, timers, time entries, timesheets, workload.

Acceptance: Time and sprint metrics are reproducible and timezone-correct.

07. Automation Engine

- Rule builder: trigger, conditions, actions.
- Assignment, status, reminders, subtasks, notification automations.
- Durable queue, retries, idempotency, logs, loop prevention.

Acceptance: Retrying a job does not create unintended duplicate tasks or alerts.

08. AI Assistant

- Natural language project/task generation and task breakdown.
- Summaries, daily planning, deadline risk and workload suggestions.
- Document assistance, permission-aware workspace retrieval.
- Action proposals, preview, explicit confirmation, usage limits.

Acceptance: AI cannot read unauthorized data or silently perform destructive actions.

09. Notifications, Search & Integrations

- Unified inbox, mentions, assignments, due dates, project activity.
- Permission-aware global search, command palette, saved queries.
- Optional calendar, drive, GitHub, webhooks, import/export integrations.

Acceptance: A notification opens the correct authorized resource and respects preferences.

10. Settings, Security & Operations

- Workspace and project settings, custom fields, permission administration.
- Audit logs, RLS, secure storage, rate limiting, backups.
- Monitoring, migrations, automated tests, CI/CD, deployment.
- Platform admin console with privacy-respecting support workflows.

Acceptance: Cross-workspace access attempts are rejected and audited where appropriate.

06 Task Lifecycle and Approval Semantics

> **Default: To Do → In Progress → Ready for Review → Completed. Rejected review returns to In Progress with feedback.**

- Statuses are configurable per project; preserve stable semantic categories for analytics.
- A review-required task cannot become completed until an authorized reviewer approves.
- Reviewer decisions store reviewer_id, timestamp, decision, comment, and task version.
- Allow multiple reviewers or sequential approvals only as a clearly specified extension; begin with one reviewer.
- Support blocked state as a flag/reason or configured status; distinguish it from ordinary progress.
- Track every status transition and important assignment in task activity history.
- Parent/subtask completion logic must avoid double counting in project progress.

Progress formula

Default: completed leaf tasks / total eligible leaf tasks × 100. Exclude archived/cancelled tasks from denominator according to a documented project setting. Advanced mode: weighted progress using story points or configured task weights. Show the selected metric and its definition in the UI.

07 Owner Command Center - Required Dashboard

The Owner Command Center is the signature feature. It aggregates only projects the current user is authorized to view. It must support cross-project navigation and actionable drill-down.

Section	Must display	Interaction
Project Portfolio	Name, status, progress, manager, due date, health	Open project / filter / sort
Executive KPIs	Active projects, total/overdue tasks, reviews, milestones	Click to filtered task list
Needs Attention	Overdue, blockers, pending approvals, at-risk milestones	Open exact issue
Team Workload	Member assignments, capacity, overdue work	Filter by project/member
Unified Activity	Recent authorized task, chat, doc, milestone events	Open linked resource
Inbox Summary	Mentions, important unread channels, assigned items	Open conversation
Timeline	Upcoming milestones and deadlines	Calendar / timeline drill-down

Role-aware dashboard defaults

- Workspace Owner: portfolio, risk, cross-project activity, reviews, workload.
- Project Manager: managed projects, assigned work, blockers, sprint health, approvals.
- Contributor: My Tasks, Today, Upcoming, Mentions, submissions and reviews.
- Viewer: authorized read-only project summaries and activity.

Filters and freshness

Support All Authorized Projects, My Projects, Managed Projects, Assigned to Me, Project, Member, Status, Date Range. Realtime updates should refresh relevant counters without expensive full-table subscriptions; maintain consistency after reconnect.

08 Communication and Notification Rules

- Creating a project creates a default #general project channel (configurable).
- Project channels belong to one project; workspace channels may be shared workspace-wide.
- Project access does not automatically imply access to a more restricted private channel.
- Project removal revokes project channel, document, file, and realtime access immediately.
- Messages support author, channel, optional thread parent, body, attachments, edits, and deletion metadata.
- Unified inbox aggregates only authorized notifications across projects.
- Unread counts are per-user and update on read; avoid duplicate notifications.
- Mention and assignment notifications deep-link to the correct workspace and resource.

09 Final 45 Primary Screens

The 45 primary screens below are a navigation baseline. Account security, preferences, and notifications are sections/tabs within consolidated settings pages rather than three additional standalone primary screens.

Public & Authentication (7)

Landing • Login • Signup • Forgot Password • Reset Password • Email Verification • Onboarding

Main Workspace (7)

Home / Owner Dashboard • Unified Inbox • My Tasks • Everything / All Tasks • Calendar • Dashboards • Global Search

Project Management (10)

Spaces Explorer • Space Overview • Folder Overview • Project Overview • Project List • Project Kanban • Project Table • Project Gantt / Timeline • Project Workload • Task Details

Team & Collaboration (5)

Chat Hub • Channel / Group Chat • Direct Messages • Team Members • Invitations

Productivity & Advanced (8)

Documents & Wiki • Document Editor • Goals & OKRs • Sprints & Backlog • Time Tracking & Timesheets • Automations • AI Assistant • Reports & Analytics

Settings & Administration (8)

Profile & Account Settings • Workspace General Settings • Members & Roles • Permissions • Custom Fields & Workflows • Integrations • Audit Logs • Trash & Recovery

Count: 7 + 7 + 10 + 5 + 8 + 8 = 45 primary screens. Additional dialogs, nested tabs, approval panels, channel settings, and platform-admin operational views are not counted as separate primary screens.

10 Navigation, UI Design and Responsive Behavior

Visual system

Token	Value / principle
Canvas	#F7F9F6
Forest / dark	#183A30 / #111C18
Primary sage	#47735C
Soft sage	#DCE7DE
Typography	Inter or a similar legible sans-serif
Components	Quiet surfaces, thin borders, moderate radius, clear hierarchy
Interaction	Hover, focus, loading, error, empty, success and disabled states
Responsive	Desktop-first; tablet collapses sidebar; mobile stacks panes

Layout rules

- Persistent workspace switcher and collapsible sidebar.
- Global search / command palette available from main app shell.
- Project header exposes Overview, Tasks, Board, Chat, Docs, Files, Timeline, Activity.
- Chat uses own messages on the right and others on the left, with group sender identity.
- Task details open as a panel/modal with deep-linkable route; keyboard accessible.
- Use consistent empty states and meaningful CTAs; never hide missing functionality behind decorative buttons.
- Support light/dark themes with accessible contrast and visible focus states.

11 Suggested Route Map

Area	Example routes
Auth	/login, /signup, /forgot-password, /reset-password, /onboarding
Workspace	/w/[slug], /w/[slug]/inbox, /w/[slug]/my-tasks, /w/[slug]/calendar
Project	/w/[slug]/projects/[projectId], /tasks, /board, /table, /timeline, /workload
Task	/w/[slug]/projects/[projectId]/tasks/[taskId]
Collaboration	/w/[slug]/chat, /w/[slug]/chat/[channelId], /w/[slug]/dm/[conversationId]
Productivity	/w/[slug]/docs, /goals, /sprints, /time, /automations, /ai
Admin	/w/[slug]/settings, /w/[slug]/members, /platform-admin

Route names are implementation recommendations; preserve existing project routes when migrating. A route inventory and redirect plan must be produced before any rename.

12 Recommended Technology and Service Boundaries

Concern	Recommendation
Web application	Next.js App Router, React, TypeScript
UI / forms	Tailwind CSS, shadcn/ui, React Hook Form, Zod
Database / identity	PostgreSQL + Supabase Auth and RLS
Realtime	Supabase Realtime with scoped authorization
File storage	Private Supabase Storage buckets with signed/authorized access
State / caching	TanStack Query; Zustand only for client UI state
Rich documents	Tiptap; Yjs or equivalent for concurrent collaboration
DnD / charts	dnd-kit, Recharts
Background work	Durable worker/queue with retries and idempotency
AI	Server-side LLM adapter, schema-validated output, approval flow
Testing	Vitest, Playwright, DB integration tests, RLS tests
Operations	Sentry or equivalent, structured logs, CI/CD, backups

Engineering rules

- Audit the existing repository before installing libraries or creating new tables.
- Prefer server actions or API handlers with explicit authorization and Zod validation.
- Avoid exposing service-role keys in client bundles.
- Use transactions for cross-table membership, task, approval, and automation changes.
- Treat realtime as a transport, not as the source of truth.
- Store timestamps in UTC and render using the user/workspace timezone.
- Use pagination and indexes for activity, messages, search, and dashboard queries.

13 Database Blueprint

Core entity families

Domain	Required entities
Identity	profiles, user_preferences, sessions via auth provider
Tenancy	workspaces, workspace_members, workspace_invitations
Organization	spaces, folders, projects, project_members, project_role_assignments
Task engine	tasks, task_assignees, task_dependencies, checklists, task_comments, task_events
Approvals	review_requests, review_decisions, workflow_definitions
Communication	channels, channel_members, messages, message_reactions, message_reads
Knowledge	documents, document_versions, file_assets, file_links
Planning	goals, key_results, milestones, sprints, time_entries
Platform	notifications, automation_rules, automation_runs, audit_logs, saved_views

Suggested key columns and constraints

- workspaces(id, slug, owner_id, name, created_at); slug unique under chosen scope.
- workspace_members(workspace_id, user_id, role, status); unique(workspace_id, user_id).
- projects(id, workspace_id, space_id, folder_id nullable, owner_id, visibility, status, due_at).
- project_members(project_id, user_id, role, job_title nullable); unique(project_id, user_id).
- tasks(id, workspace_id, project_id, parent_task_id nullable, status_id, title, due_at, priority, version).
- task_assignees(task_id, user_id); unique(task_id, user_id); assignee must be authorized for the task project.
- channels(id, workspace_id, project_id nullable, visibility, type); project_id nullable for workspace channels/DMs.
- messages(id, channel_id, author_id, parent_message_id nullable, body, created_at, edited_at).
- review_requests(id, task_id, reviewer_id, state, submitted_at); review_decisions(request_id, reviewer_id, decision, comment, decided_at).
- activity_events(id, workspace_id, project_id nullable, actor_id, resource_type, resource_id, action, occurred_at).

Integrity rules

- Every project must belong to exactly one workspace.
- A task and its parent task must belong to the same project.
- A project member must have valid workspace membership unless an explicit guest policy is implemented.
- Channel membership and resource visibility are independently checked.
- Use database constraints/triggers where appropriate to prevent cross-workspace references.
- Keep immutable audit events for sensitive role, permission, and approval changes.

14 Permission Enforcement and Threat Model

Required authorization checks

- Read/write checks for workspaces, projects, tasks, docs, messages, files, search, exports.
- RLS policies for all user-owned and tenant-owned tables.
- Server authorization before issuing signed file URLs.
- No cross-project leak via counts, global search, activity feed, notifications, or AI prompts.
- Revoke permissions promptly when membership is removed; avoid stale client authorization.
- Protect invite tokens, OAuth credentials, integration secrets, and webhook signatures.
- Rate limit auth, invitations, messages, AI, automation, and expensive search endpoints.
- Log sensitive role changes, deletions, approvals, and administrative actions.

Must-pass negative tests

Attempt to read Project B as Project A-only member; fetch another workspace file; subscribe to a private channel; search unauthorized documents; approve a task without reviewer permission; alter project_id in a request; access private data as a platform operator. All must be denied unless explicitly authorized by a tested policy.

15 Automation and AI Safety Architecture

Automation pipeline

Domain event → matching rules → condition evaluation → durable job → idempotent action → execution log → retry or failure report. Implement a recursion limit and loop detection for rules that emit further events.

AI proposal pipeline

User request → authorized context retrieval → schema-validated plan → preview/diff → user confirmation → server-side permission recheck → transactional write → audit event. Never execute destructive changes from free-form model text alone.

AI privacy rules

- Retrieve only content authorized for the current user and workspace.
- Mask secrets and avoid passing credentials or unnecessary personal data to AI providers.
- Provide cancellation and error handling for failed generations.
- Track usage, rate limits, and explicit user confirmation for actions.

16 Implementation Roadmap and Deliverables

Phase 0 - Audit & Design

Inventory existing code, routes, tables, UI, RLS, tests; define architecture and migration plan.

Exit artifact: Architecture decision record, route map, schema proposal, gap report.

Phase 1 - Identity & Tenancy

Auth, workspace creation, switching, membership, invitations, roles, RLS.

Exit artifact: Functional workspace lifecycle + cross-tenant tests.

Phase 2 - Project & Task Core

Projects, project membership, task CRUD, dependencies, reviews, activity history.

Exit artifact: Full owner → manager → contributor task workflow.

Phase 3 - Views & Dashboards

List/Kanban/Table/Calendar/Timeline, owner command center, reporting.

Exit artifact: Consistent progress and permission-aware aggregation.

Phase 4 - Collaboration

Project channels, DMs, comments, files, notifications, realtime.

Exit artifact: Project isolation, unread states, live messaging.

Phase 5 - Productivity

Docs/wiki, goals, sprints, time, workload and calendar.

Exit artifact: Persisted workflows and correct calculations.

Phase 6 - Automation & AI

Rule engine, durable workers, AI proposals and summaries.

Exit artifact: Idempotency, approval, logs, error recovery.

Phase 7 - Hardening & Launch

Accessibility, performance, RLS audit, E2E, monitoring, backups, deploy.

Exit artifact: Production readiness report and release checklist.

Phase gate policy

Do not move to the next phase with broken core flows, unreviewed migrations, failing security tests, or nonfunctional UI controls. Each phase must include a demo walkthrough, automated test results, and known limitations.

17 End-to-End Acceptance Test Matrix

ID	Scenario	Pass condition
T01	Signup → verify → create workspace	Workspace persists and owner role assigned.
T02	Invite user → accept → join workspace	Membership state correct; unauthorized invite rejected.
T03	Owner creates three projects	Projects appear in portfolio and navigation.
T04	Different members per project	Private data isolated by project.
T05	Same member, different project roles	Permissions follow project context.
T06	Contributor updates assigned task	All views and owner dashboard update.
T07	Submit → review → reject → resubmit → approve	History, reviewer, status and notifications correct.
T08	Chat in Project A	Project B-only member cannot read it.
T09	Remove project member	Task, chat, file and realtime access revoked.
T10	Cross-project dashboard filter	Only authorized projects counted.
T11	Task dependencies / recurring task	No cycles or duplicate recurrence.
T12	Concurrent doc edits	No silent lost updates.
T13	Automation retry	No duplicate side effects.
T14	AI task creation proposal	No write before confirmation.
T15	Restore archived/deleted task	Correct state and audit history.
T16	Responsive and keyboard navigation	Core workflows accessible at common viewports.

18 Definition of Done and Release Checklist

- Real database persistence with reviewed migrations and seeded demo data isolated from production.
- Server-side and RLS authorization verified, including negative tests.
- Functional UI states: loading, empty, error, success, disabled, offline/reconnect where relevant.
- Consistent data after mutations across dashboards, views, chat, notifications, and search.
- Accessible controls, responsive layout, keyboard navigation, and sufficient contrast.
- Integration and E2E test suites pass; no critical defects.
- Performance budgets and pagination applied to large datasets.
- Auditability, backups, logging, error tracking, and secure secret handling in place.
- Documentation: environment setup, migrations, permissions, API contracts, deployment, troubleshooting.
- A final report lists implemented features, changed files, test results, limitations, and next steps.

19 Copy-Ready Google Antigravity Master Prompt

Use the following instructions with the repository and this PDF attached. Execute one phase at a time; do not claim completion without tests.

- You are the lead full-stack architect and engineer for OneDesk. Treat this PDF as the source of truth for product behavior and acceptance criteria.
- FIRST: inspect the current repository. Report the existing framework, routes, database schema, auth, RLS, reusable UI components, integrations, broken flows, and gaps against this blueprint. Do not overwrite working features or create duplicates.
- THEN: propose a phase-by-phase implementation plan with database migrations, route changes, permission design, and test strategy. Identify ambiguous requirements before irreversible changes.
- BUILD: a production-oriented multi-tenant project-centric work management platform. Users must create workspaces and multiple projects, assign different project teams and roles, manage tasks and approvals, collaborate via project-specific chat/docs/files, and see authorized projects on one central dashboard.
- ENFORCE: workspace isolation, project-level RBAC, channel/file permissions, secure API boundaries, RLS, validation, rate limiting, and audit logs. Do not rely on frontend route protection alone.
- IMPLEMENT: real database persistence, responsive premium Alpine Sage + Obsidian Forest UI, realtime updates, functional navigation, meaningful empty/error states, and accessible interactions.
- DO NOT BUILD: CRM, client portals, billing, subscriptions, payments, accounting, sales pipelines, or financial tracking.
- FOR EACH PHASE: show files changed, migrations, features implemented, manual test steps, automated tests, security checks, known issues, and remaining work. Run relevant tests and fix regressions before marking complete.
- IMPORTANT: do not fabricate completed functionality, fake analytics, or substitute static mockups for real working features. Obtain confirmation before destructive migrations or irreversible changes.

20 Final Product Acceptance Statement

OneDesk is complete only when an owner can create multiple projects with different teams, contributors can perform and submit assigned work, managers can review it, project collaboration remains isolated, and all authorized progress appears accurately in one central dashboard.

This specification is the baseline for UI design, database schema, engineering implementation, security review, and release testing. The 45-screen count is a navigation planning unit, not a limit on components, dialogs, or administrative tools.

Recommended handoff: provide this PDF to Google Antigravity alongside the current repository. Ask it to complete Phase 0 (audit) and return the findings before making structural changes.